

etcd Backup and Restore

Why, what to think about first, how to back up, how to automate it, how to restore, and how to prove the restore really worked

THE WHOLE IDEA IN ONE LINE

etcd is the cluster's memory. A backup is a photocopy of that memory. A restore is replacing the memory with the photocopy. **A backup you've never restored is a hope, not a backup.**

Contents

1. Why back up: what's in etcd and what isn't
2. Think before you back up: the checklist
3. How to back up (commands, explained)
4. Automating it: script, service, timer
5. How to restore, step by step
6. How to validate a restore
7. Common mistakes
8. Quiz and answer key

etcd 3.6.6 (etcdctl + etcdctl) · stacked etcd on cka-m · restore drill done 2026-09-24

1. Why back up

1.1 The restaurant register

Imagine a restaurant where nobody remembers anything. The **register book** at the counter lists every order, table and staff member, and everyone looks things up there.

- **etcd** = the register book.
- **API server** = the only person allowed to write in it or read from it.
- **Controllers, scheduler, kubelets** = staff who keep asking "what does the register say?" and make reality match it.

If the register burns, containers may keep running for a while, but **nobody knows what should exist anymore**. That's a dead cluster, and it's exactly what happened to us on 2026-09-23 (study notes 3).

1.2 What's in etcd, and what isn't

In etcd (a restore brings it back)	NOT in etcd (a restore does NOT bring it back)
Every API object: Namespaces, Deployments, Pods, Services, ConfigMaps, Secrets , RBAC, CRDs, Nodes, Leases	Data inside PersistentVolumes (database rows, uploads): back those up separately
The desired state ("3 replicas of X")	Container images (they're in the registry)
	The certificates and keys in <code>/etc/kubernetes/pki</code>

A SNAPSHOT IS AS SENSITIVE AS ALL YOUR PASSWORDS TOGETHER

Secrets are stored in etcd only base64-encoded (unless encryption at rest is on), so anyone who can read a snapshot can read every Secret. Treat backup files accordingly.

2. Think before you back up: the checklist

#	Question	Our answer	Why it matters
1	Which tool?	etcdctl / etcdutl 3.6.6 , same as the server	In 3.6, <code>snapshot status/restore</code> moved from etcdctl to etcdutl
2	How do I connect?	Endpoint + 3 TLS files, read from <code>etcd.yaml</code>	etcd only talks to clients with a certificate from its own CA
3	What else to save?	<code>/etc/kubernetes/pki</code>	The snapshot is the data ; the PKI is the identity
4	Who can read the backup?	root only (<code>umask 077</code> , folder 700)	Every Secret + the CA private key
5	Where is it stored?	On cka-m and copied to Windows	A backup on the same machine dies with it. 3-2-1 rule : 3 copies, 2 media, 1 off-site
6	How often, how long?	Every 6 h, keep 7 days (28 snapshots)	How much can we afford to lose? (RPO = 6 h)
7	Who remembers?	A systemd timer	Manual backups get forgotten; a CronJob would die with the cluster
8	Is the file good?	Verified <i>before</i> it gets its final name	A corrupt backup found on restore day is worse than none
9	Ever restored it?	Yes, drill on 2026-09-24	Untested backups are hope

2.1 Why the PKI matters: the ID-card analogy

Every component carries an ID card (certificate) stamped by the cluster's CA. If cka-m dies and you run `kubeadm init` on a new VM, kubeadm creates a **new CA**, a new stamp. The workers and your Windows kubeconfig carry cards with the **old** stamp, so nobody trusts anybody: nodes go NotReady, kubectl fails, ServiceAccount tokens stop validating.

THE FIX INTERVIEWERS WANT TO HEAR

Put the backed-up `/etc/kubernetes/pki` back on the new node **before** `kubeadm init`. kubeadm reuses certificates it finds, so the cluster keeps its identity.

3. How to back up

3.1 Find the connection details (never guess)

```
sudo grep -E -- '--(listen-client-urls|trusted-ca-file|cert-file|key-file)='  
/etc/kubernetes/manifests/etcd.yaml
```

Flag	Our value	Plain meaning
--endpoints	https://127.0.0.1:2379	Where etcd listens
--cacert	/etc/kubernetes/pki/etcd/ca.crt	Which stamp I trust (to check etcd's card)
--cert	/etc/kubernetes/pki/etcd/server.crt	My ID card
--key	/etc/kubernetes/pki/etcd/server.key	Proves the card is mine

3.2 Take the snapshot

```
sudo etcdctl --endpoints=https://127.0.0.1:2379 \  
--cacert=/etc/kubernetes/pki/etcd/ca.crt \  
--cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key \  
snapshot save /var/backups/etcd/etcd-$(date +%F-%H%M).db
```

This asks the **running** etcd for a consistent copy; no downtime is needed. etcdctl writes `<name>.part` first and renames it when the download completes.

3.3 Check it

```
sudo etcdctl snapshot status /var/backups/etcd/etcd-2026-09-24-1630.db -w table  
|  HASH  | REVISION | TOTAL KEYS | TOTAL SIZE | VERSION |  
| 62bd7044 | 10111 | 532 | 4.4 MB | 3.6.0 |
```

Column	Meaning
HASH	A fingerprint of the content, checked by etcdctl. A damaged file makes the command fail
REVISION	etcd's change counter: every write adds 1. Ours grew by ~2,600 in 28 minutes with nothing deployed (node heartbeats every 10 s, status updates, events)
TOTAL KEYS	Number of stored objects
VERSION	Storage format version, always <code>x.y.0</code> (the binary is 3.6.6)

3.4 Save the PKI

```
(umask 077; sudo tar czf /var/backups/etcd/pki-$(date +%F).tgz -C /etc/kubernetes pki)
```

Our first PKI tarball was created world-readable (644), a mistake we fixed immediately. That's why `umask 077` is now in every command and in the script.

4. Automating it

File (in <code>infra/etcd-backup/</code>)	Role	Analogy
<code>etcd-backup.sh</code>	What to do	The recipe
<code>etcd-backup.service</code>	How to run it: as root, once (<code>Type=oneshot</code>), low priority	The cook
<code>etcd-backup.timer</code>	When : every 6 h (<code>OnCalendar=*-*-* 00/6:00:00</code>); <code>Persistent=true</code> runs a missed backup at boot	The alarm clock

4.1 What the script does



WHY VERIFY BEFORE RENAMING?

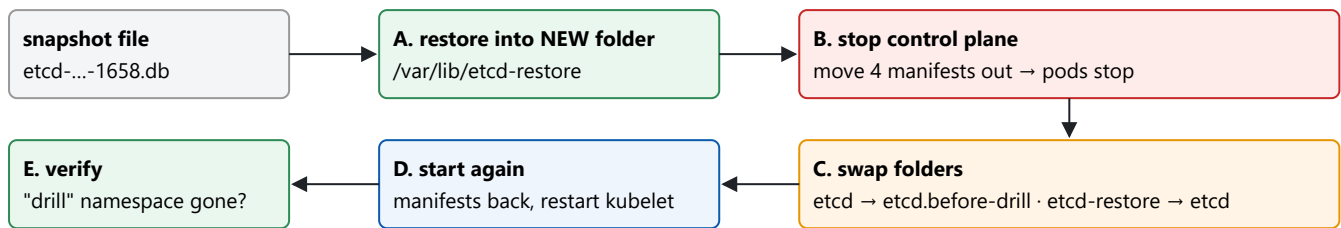
Pruning keeps "the newest 28 `.db` files". If damaged snapshots also got the `.db` name, a week of silent failures would push **every good snapshot** out, and you'd only find out on restore day. So only a verified file gets the final name.

4.2 Why a systemd timer, not cron or a CronJob?

- **vs cron:** `Persistent=true` catches up on runs missed while the VM was off; output goes to the journal; `systemctl list-timers` shows the last and next run.
- **vs a Kubernetes CronJob:** a backup of the cluster must not depend on the cluster. The day the scheduler or API server is broken is the day you need the backup.

```
sudo systemctl start etcd-backup.service           # run once by hand first
sudo journalctl -u etcd-backup -n 30 --no-pager     # status table + "OK: ..."
systemctl list-timers etcd-backup.timer --no-pager # next run
```

5. How to restore



API server is DOWN from B to D (kubectl fails: expected)

The drill: snapshot at 16:58 → create namespace "drill" → restore 16:58 → "drill" must be gone

Step 0: read the member's identity (never guess)

```
sudo grep -E -- '--(name|data-dir|initial-cluster|initial-advertise-peer-urls)='  
/etc/kubernetes/manifests/etcd.yaml  
sudo grep -B1 -A2 'path: /var/lib/etcd' /etc/kubernetes/manifests/etcd.yaml
```

A restore creates a **brand-new database** that records **who its member is**. That identity must match what the etcd pod says about itself; otherwise etcdctl uses its defaults (default at `http://localhost:2380`) and etcd breaks.

Flag	Plain meaning	Ours
<code>--name</code>	"My name is..."	cka-m
<code>--initial-advertise-peer-urls</code>	"Other members reach me at..."	https://10.0.0.100:2380
<code>--initial-cluster</code>	"The cluster consists of..."	cka-m=https://10.0.0.100:2380
<code>--data-dir</code>	Folder inside the container	/var/lib/etcd
<code>hostPath.path</code>	Folder on the node's disk : this is what we swap	/var/lib/etcd

Step A: restore into a new folder (the live cluster isn't touched)

```
sudo etcdctl snapshot restore /var/backups/etcd/etcd-2026-09-24-1658.db \  
  --data-dir=/var/lib/etcd-restore --name=cka-m \  
  --initial-cluster=cka-m=https://10.0.0.100:2380 \  
  --initial-advertise-peer-urls=https://10.0.0.100:2380 \  
  --bump-revision=1000000000 --mark-compacted  
# log: bumping latest revision 12718 → 1000012718; marking revision compacted
```

--BUMP-REVISION AND --MARK-COMPACTED IN PLAIN WORDS

Every controller remembers "the last change number I saw". The live cluster had moved past the snapshot's number. If the restored etcd started *lower*, controllers would think nothing changed and keep stale data in memory. Jumping the counter far ahead and marking old history as gone forces every controller to reload fresh data.

Steps B–D: stop, swap, start

```
# B. stop ALL four static pods (they keep cluster state in memory)
sudo mkdir /etc/kubernetes/manifests-stopped
sudo mv /etc/kubernetes/manifests/*.yaml /etc/kubernetes/manifests-stopped/
sudo crictl ps --name '^(etcd|kube-apiserver|kube-controller-manager|kube-scheduler)$' # repeat
until empty

# C. swap (keep the old folder: it's your way back)
sudo mv /var/lib/etcd /var/lib/etcd.before-drill
sudo mv /var/lib/etcd-restore /var/lib/etcd

# D. start again
sudo mv /etc/kubernetes/manifests-stopped/*.yaml /etc/kubernetes/manifests/
sudo rmdir /etc/kubernetes/manifests-stopped
sudo systemctl restart kubelet
```

WHY SWAP FOLDERS INSTEAD OF EDITING ETCD.YAML?

The common exam shortcut is to point `hostPath` at `/var/lib/etcd-restore`. It works today, but a later `kubeadm upgrade` regenerates `etcd.yaml` with `/var/lib/etcd`, and would silently switch you back to the **old** data. Swapping keeps the manifest exactly as `kubeadm` made it. Know both methods.

Rollback if the API server isn't back within 5 minutes: repeat B, move the restored folder aside, move `etcd.before-drill` back to `/var/lib/etcd`, repeat D. Don't try random fixes on a cluster that's down.

6. How to validate a restore

#	Check	Command	What we saw
1	Control plane back, no crash loops	<code>sudo crictl ps --name '^(etcd kube-...)\$'</code>	All 4 Running, attempt 0
2	The proof: the change made after the snapshot is gone	<code>kubectl get ns</code>	drill missing ✅
3	Nodes and pods healthy	<code>kubectl get nodes, get pods -A</code>	3 Ready, all Running
4	etcd healthy, right identity	<code>etcdctl ... endpoint status -w table</code>	Member ID 99c2fd4fe11e28d9 = the ID the restore wrote
5	Writes work	<code>kubectl create ns write-test && kubectl delete ns write-test</code>	OK
6	Backups work on the restored etcd	<code>systemctl start etcd-backup.service</code>	Revision 1000013198: the bump is live
7	Remote clients still trusted	Windows: <code>kubectl get nodes</code>	3 Ready
8	Only then clean up	<code>sudo rm -rf /var/lib/etcd.before-drill</code>	It holds every Secret

Our honest slip: the old folder was deleted before check 7 had run. The risk was small (the PKI never changed), but the rule is: **the rollback copy is the last thing you delete**.

Other things we noticed: `calico-kube-controllers` restarted once (it lost the API server during the swap), which is expected. The DB was 4.4 MB with only 1.9 MB in use, because `--mark-compacted` freed history; `etcdctl defrag` would give the space back.

REAL-WORLD CAUTION

A restore rolls back **everything** after the snapshot, not only your mistake. Pods created later become unknown to Kubernetes, and IP allocations can clash. Restore for disasters, using the newest good snapshot.

7. Common mistakes

Mistake	What happens
Backups only on the control-plane node	They die with the node
Snapshot saved without the PKI	A rebuilt node gets a new CA, and nothing trusts anything
Restore without <code>--name / --initial-cluster / peer URL</code>	Member identity mismatch; etcd fails or records a wrong member
Restoring on top of the existing <code>/var/lib/etcd</code>	<code>etcdctl</code> refuses (data dir exists), or you destroy your way back
API server left running during the swap	Stale cached data, or writes into the wrong database
Backup files readable by everyone	Anyone on the node reads every Secret and the CA key
<code>etcdctl snapshot restore</code> on 3.6	Removed: use <code>etcdctl</code>
Never testing a restore	You find out on the worst day that it doesn't work

8. Quiz

1. What does etcd store, and name two important things it does NOT store.
2. Why is a snapshot file as sensitive as all your passwords?
3. Why do we back up `/etc/kubernetes/pki` together with the snapshot?
4. If cka-m died and you rebuilt it, when exactly do you put the PKI back, and why?
5. Our backups live in `/var/backups/etcd` on cka-m. Name two failures they won't survive.
6. What's the 3-2-1 rule?
7. Why does the script verify the snapshot before renaming it from `.tmp` to `.db`? Link your answer to pruning.
8. Why a systemd timer and not a Kubernetes CronJob?
9. What does `Persistent=true` do, and why does it matter in our lab?
10. Which tool takes the snapshot, and which tool checks and restores it, in etcd 3.6?
11. What does the REVISION number mean, and why did ours grow with nothing deployed?
12. Why must a restore get `--name`, `--initial-cluster` and `--initial-advertise-peer-urls`?
13. What's the difference between `--data-dir` and the `hostPath` in `etcd.yaml`?
14. Explain `--bump-revision` and `--mark-compacted` in plain words.
15. Why stop all four static pods, not only etcd?
16. Why swap folders instead of pointing `hostPath` at the restore folder?
17. How did we *prove* the restore worked?
18. When may you delete `/var/lib/etcd.before-drill`, and why delete it at all?

Answer key

1. All API objects (incl. Secrets, RBAC, CRDs, Nodes). Not: data inside PersistentVolumes, container images, the PKI files.
2. Secrets are stored only base64-encoded (unless encryption at rest is enabled), so the snapshot contains them in readable form.
3. The snapshot is the data; the PKI is the identity. Without the original CA, a rebuilt control plane isn't trusted by nodes and clients.
4. **Before** `kubeadm init`: kubeadm reuses existing certificates, so the cluster keeps its CA and everyone still trusts it.
5. E.g. the VM disk dies or is corrupted; the VM is deleted or reinstalled; an accidental `rm -rf` or ransomware on the node.
6. 3 copies, on 2 different media, 1 of them off-site.
7. Pruning keeps only the newest 28 `.db` files. If corrupt snapshots were named `.db`, a run of failures would push the last good ones out of retention.
8. A backup of the cluster must not depend on the cluster's own scheduler and API server being healthy.
9. It runs a missed scheduled run as soon as the machine boots. Our VMs are often off at 00/06/12/18 h.
10. `etcdctl snapshot save` takes it; `etcdctl snapshot status` / `restore` check and restore it.
11. etcd's global change counter: every write adds 1. Node heartbeats (every ~10 s), status updates and events keep writing.
12. The restore writes a new member list. The identity must match the etcd pod's flags, or etcd fails or records a wrong member (defaults are `default` / `localhost`).
13. `--data-dir` is the path inside the container; `hostPath` is the folder on the node's disk mounted there, which is what you swap.
14. Controllers remember the last revision they saw, which is higher than the snapshot's. Jumping the counter ahead and marking history compacted forces them to reload everything instead of trusting stale caches.
15. The API server, controller-manager and scheduler keep cluster state in memory; they must restart to reload it from the restored etcd.
16. So `etcd.yaml` stays identical to what kubeadm generated; a later `kubeadm upgrade` would otherwise switch back to `/var/lib/etcd`, the old data.
17. We created namespace `drill` after the snapshot; after the restore it was gone. Plus: member ID matched, writes worked, a new backup succeeded, Windows kubectl worked.
18. Only after every check has passed. Delete it because it contains a full copy of the cluster, including every Secret.